

Yocto

Author: Brian Kang
Create: 5/13/2026
Last Modified: 5/19/2026

Preface

In modern embedded systems engineering, the Yocto Project has become the industry standard for constructing reliable, reproducible, and highly optimized Linux Board Support Packages (BSPs). However, transitioning an advanced multimedia pipeline from a vendor's reference design to a production-ready product requires a deep, structural understanding of how Yocto isolates, compiles, and packages software.

This technical note serves as a practical, hands-on guide to mastering the Yocto Project build environment, using the Renesas R-Car H3SK SoC as a reference platform. Drawing from extensive real-world experience in video subsystem development and hardware-software co-design, this document bridges the gap between theoretical build systems and daily engineering workflows.

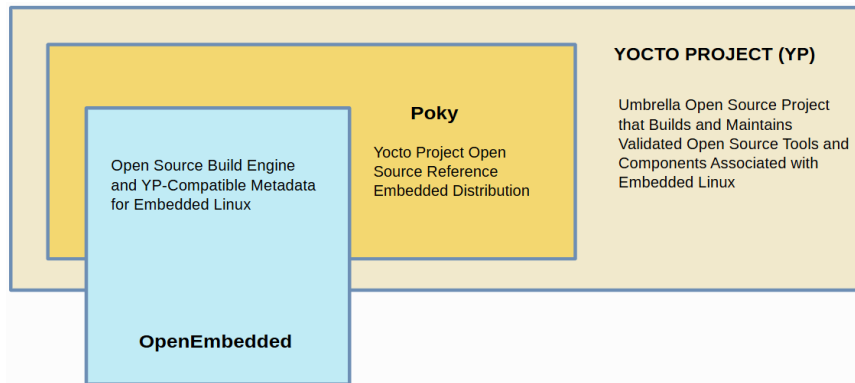
Readers will be guided through a complete system lifecycle: from establishing an optimized Ubuntu 22.04 Host and decoupled SDK development environment, to decoding BitBake's internal logging mechanics and manipulating core multimedia recipes like v4l-utils. Special emphasis is placed on moving away from brittle, manual source patches toward clean, scalable meta-layer architectures that seamlessly bundle custom video bridge drivers, device trees, and V4L2 framework extensions. Whether you are conducting initial board bring-up or streamlining customer integration pipelines, this note delivers the exact technical paradigms required to accelerate your embedded development lifecycle.

1. Yocto

Yocto is an umbrella Open Source Project that Builds and maintains the open source tools and components associated with Embedded Linux.

Yocto helps developers create custom Linux-based systems that are designed for embedded products regardless of the product's hardware architecture.

It can generate Linux Kernel Image, U-Boot, root filesystem, your applications, and SDK.

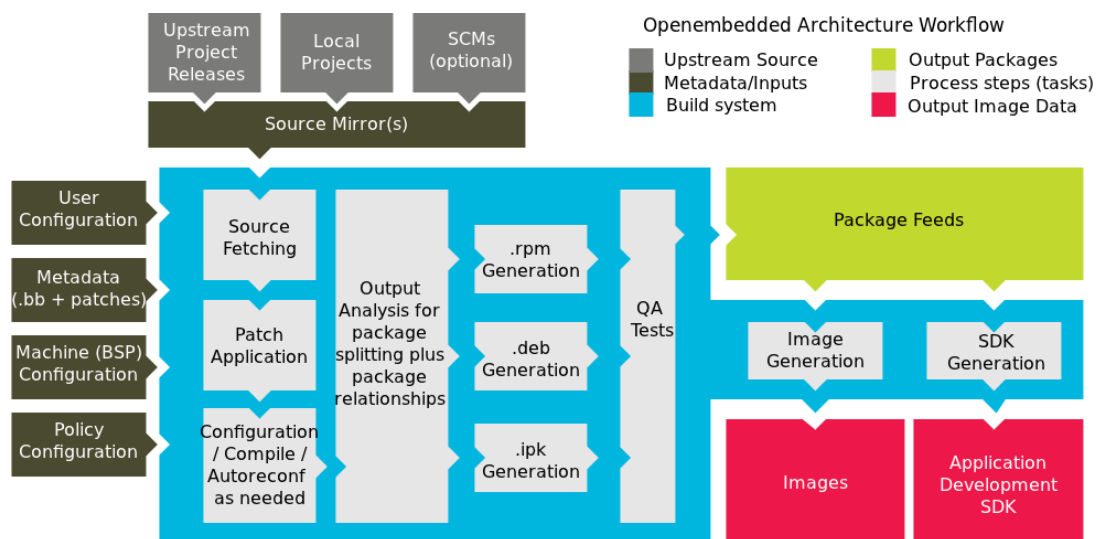


Yocto is consisted of OPENEMBBEDED and Poky.

OpenEmbedded (OE): An open-source framework designed for building embedded Linux systems. It consists of BitBake—the engine that performs the actual build process—and OpenEmbedded-core (oe-core), a vast collection of "recipes" that specify how software components are to be built.

Poky is a reference sample build system provided by the Yocto Project. Rather than being a distinct, standalone engine, it is akin to a "complete package"—it adopts the core elements of OpenEmbedded and pre-configures them, allowing users to immediately test the build process.

1.1 Workflow



Steps	Description
-------	-------------

recipes	Read recipes and configuration file
Fetch	get the source code
Extract	unpack the sources
Patch	apply patches for bug fixes and new capability
Configure	set up your environment specifications
Build	compile and link
Install	copy files to target directories
Package	bundle files for installation
Image	Repeat upper steps, and then, generates Images and SDK.

1.2 Yocto Project Components

Bitbake

BitBake is a generic task execution engine that allows shell and Python tasks to be run efficiently and in parallel while working within complex inter-task dependency constraints. other word, it is a Task executor and scheduler.

BitBake is a program written in the Python language.

BitBake interprets metadata, decides what tasks are required to run, and executes those tasks.

Recipes

Details about particular pieces of software. It includes, the location from which to download the source, any source patches to be applied to that source (if needed), which special configuration options to apply, how to compile the source files, and how to package the compiled output.

Classes

An abstraction of common build information (e.g. how to build a Linux kernel).

Class files contain information that is useful to share between metadata files.

Configuration

Configuration files define various configuration variables that govern the project's build process.

Configuration data acts as the glue to bind everything together.

Layers

Layers are repositories that contain related metadata (i.e. sets of instructions) that tell the OpenEmbedded build system how to build a target.

Layers allow you to isolate different types of customizations from each other.

Some Basic Terms

Terms	Description
Configuration Files	Global definitions. It tells the build system what to do
append files	Files that append build information to a recipe file. It has "bbappend"
Metadata	It is used to construct a Linux distribution, files that build system parses when it

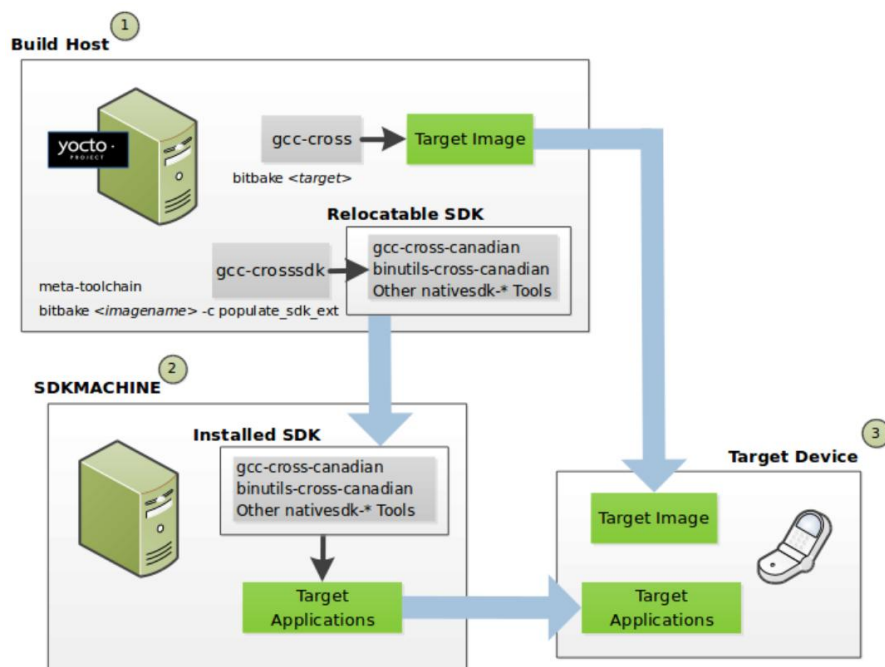
	need to build an image
OpenEmbedded Core	It is meta-data composed of foundation recipes, classes and associated files
Poky	Reference embedded Linux Distribution. Poky is a reference embedded distribution and a reference test configuration. Poky is not a product level distro. Rather, it is a good starting point for customization.
Build System	Bitbake. scheduler and execution engine
Build Directory	Main directory. The TOPDIR variable points to the Build Directory. By default, the Build Directory contains TMPDIR, which is a temporary directory the build system uses for its work.
Packages	The output of build used to create final image
BSP	Board Support Package. A group of drivers, definitions, and other components that provide support for a specific hardware configuration.
Custom SDK	Software Development Kit with Compiler, libraries for the application develop.
Image	A binary form of Linux distribution

1.3 Board Support Package

BSP is a software support package required to drive a specific hardware board.

It is a bridge between Hardware and OS, and supports U-Boot, Kernel and Device, Device Tree, and Root File System. It is provided by separate meta-layer

So the final result of Yocto is Poky + BSP + Target Application provided by customer.



The host machine runs **Ubuntu 22.04 LTS** and is used to build the target image. To streamline the development process, the SDK is installed on the same host machine within a separate, dedicated Bash environment, allowing for rapid Target Application development.

It needs to open 3 Terminal Window for below purposes.

Terminal	Purpose	Core Action
BSP	To build the Yocto BSP	Copy the build results from the <code>deploy</code> directory to the TFTP and

		NFS server directories.
SDK	To build applications	Cross-compile the application, copy the output binary to the NFS directory, and execute it on the Target.
Target	Target console access	Connect to the target console using Minicom with the following serial settings: 115200//8/M/1 without FlowControl

2 Build Target Image

Your SoC provider provides BSP for their Evaluation boards.

Typically, engineers encounter Yocto through the BSP provided by their SoC vendor. While the exact setup can vary depending on the specific SoC environment, this guide will explain how to install and use Yocto based on my experience with the Renesas R-Car H3SK.

The reference is: <https://elinux.org/R-Car/Boards/Yocto-Gen3/v5.9.0>

2.1 Prepare Host

2.1.1 Install Host Packages

```
$ sudo apt-get install gawk wget git-core diffstat unzip texinfo gcc-multilib \
build-essential chrpath socat cpio python3 python3-pip python3-pexpect \
xz-utils debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa \
libssl1.2-dev pylint3 xterm
$ sudo apt-get install ncurses-dev meld curl
```

2.1.2 Python3

Yocto using Python3. If your “python” was linked to python2, follow-up below steps.

```
$ cd /usr/bin
$ sudo rm python
$ sudo ln -s python3 python
```

2.2 Build your target Image

2.2.1 Prepare WORK directory

```
$ mkdir your_project
$ cd your_project
$ export WORK=`pwd`
```

Now you can use “\$WORK” bash variable.

2.2.2 Clone and checkout branches

```
cd $WORK
git clone git://git.yoctoproject.org/poky
git clone git://git.openembedded.org/meta-openembedded
git clone https://github.com/renesas-r-car/meta-renesas
cd $WORK/poky
git checkout -b tmp 74b22db6879b388d700f61e08cb3f239cf940d18
cd $WORK/meta-openembedded
git checkout -b tmp 814eec96c2a29172da57a425a3609f8b6fcc6afe
cd $WORK/meta-renesas
git checkout -b tmp db07324cef9f09cfccafb36f05b8159421057af5
```

And then, download multimedia packages if custom BSP is required.

For example, R-CAR needs below file to download and unzip.

"<https://www.renesas.com/us/en/application/automotive/r-car-h3-m3-h2-m2-e2-documents-software>"

```
$ ls -l $WORK/../proprietary/*.zip
R-Car_Gen3_Series_Evaluation_Software_Package_for_Linux-20220121.zip
R-Car_Gen3_Series_Evaluation_Software_Package_of_Linux_Drivers-20220121.zip
```

Execute below commands to unzip and install the packages.

```
export PKGS_DIR=$WORK/../proprietary
cd $PKGS_DIR
unzip -qo R-Car_Gen3_Series_Evaluation_Software_Package_for_Linux-20220121.zip
unzip -qo R-Car_Gen3_Series_Evaluation_Software_Package_of_Linux_Drivers-20220121.zip
cd $WORK/meta-renesas
sh meta-rcar-gen3/docs/sample/copyscript/copy_proprietary_softwares.sh -f $PKGS_DIR
unset PKGS_DIR
```

2.2.3 Init build environment

```
$ source $WORK/poky/oe-init-build-env
```

The default dir is "build"

2.2.4 Update the default configurations

```
cp $WORK/meta-renesas/meta-rcar-gen3/docs/sample/conf/h3ulcb/pokygcc/mmp/*.conf ./conf/
cd $WORK/build
cp conf/local-wayland.conf conf/local.conf
```

2.2.5 Start the build

```
$ bitbake core-image-weston
```

Result is

```
./tmp/deploy/images/h3ulcb
Image
core-image-weston-h3ulcb.tar.bz2
r8a77951-ulcb.dtb
u-boot*
```

2.2.6 Build SDK

```
$ bitbake core-image-weston-sdk -c populate_sdk
```

To install SDK

```
$ ./tmp/deploy/sdk/poky-glibc-x86_64-core-image-weston-aarch64-toolchain-*.sh
```

Select "**/opt/poky/3.1.11-h3-weston**" for SDK location.

2.2.7 Rebuild

```
$ cd your_project
$ export WORK=`pwd`
$ source $WORK/poky/oe-init-build-env
$ bitbake core-image-weston
```

2.3 Prepare working Environment – TFTP and NFS

I will use below bash variables. Open new Terminal and execute below.

```
export WORK=~/rcar/50900/h3-weston
export TFTP_LOC=/srv/tftp/50900-h3-weston
export NFS_LOC=/srv/nfs/50900-h3-weston
export SDK_LOC=/opt/poky/3.1.11-h3-weston
export KERNEL_LOC=$WORK/build/tmp/work-shared/h3ulcb/kernel-source
export IMAGE_LOC=$WORK/build/tmp/deploy/images/h3ulcb
```

2.3.1 TFTP

TFTP is to download Linux Kernel and device tree from your Host to Target machine.

Install TFTP.

```
$ sudo mkdir -p /srv/tftp
$ sudo apt-get install -y tftp tftpd-hpa
$ sudo gedit /etc/default/tftpd-hpa

# /etc/default/tftpd-hpa
TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/srv/tftp"
TFTP_ADDRESS=":69"
TFTP_OPTIONS="--secure --create"

$ sudo service tftpd-hpa restart
```

Note: Without "--create", you have to reboot whole system when you update Image and ".dtb" files.

Copy "Image" and ".dtb" files from \$IMAGE_LOC to \$TFTP_LOC.

```
$ sudo mkdir -p $TFTP_LOC
$ sudo cp $IMAGE_LOC/Image $TFTP_LOC
$ sudo cp $IMAGE_LOC/r8a779*-ulcb.dtb $TFTP_LOC
```

It will copy "r8a77951-ulcb.dtb" and "r8a779m1-ulcb.dtb".

Make a symbolic link

```
$ cd /srv/tftp
$ sudo rm -f Image
$ sudo ln -s 50900-h3-weston/Image Image
$ sudo rm -f r8a779m1-ulcb.dtb
$ sudo ln -s 50900-h3-weston/r8a779m1-ulcb.dtb r8a779m1-ulcb.dtb
$ sudo rm -f r8a77951-ulcb.dtb
$ sudo ln -s 50900-h3-weston/r8a77951-ulcb.dtb r8a77951-ulcb.dtb
```

Confirm

```
$ cd /tmp
```

```
$ tftp localhost
tftp> get Image
Received 33801975 bytes in 1.4 seconds
tftp> quit

$ ls
Image

$ rm Image
$ cd ~
```

2.3.2 NFS

NFS is a network File System, it means the Linux File System is on your Host not on the Target.

Install

```
sudo apt-get install -y nfs-kernel-server nfs-common
sudo mkdir -p /srv/nfs/50900-h3-weston
sudo tar -xjf $IMAGE_LOC/core-image-weston-h3ulcb.tar.bz2 -C /srv/nfs/50900-h3-weston
sync
sudo gedit /etc/exports
/srv/nfs/50900-h3-weston *(rw,sync,no_subtree_check,no_root_squash)

sudo service nfs-kernel-server restart
```

To confirm

```
$ showmount -e localhost
Export list for localhost:
/srv/nfs/50900-h3-weston *
```

2.4 Connect Console

To find out the serial, use dmesg command.

Connect USB cable between your Host PC, and CN12 on H3SK, and type "dmesg".

```
[ .495379] usb 2-1.3: new full-speed USB device number 5 using ehci-pci
[ .610401] usb 2-1.3: New USB device found, idVendor=0403, idProduct=6001
[ .610409] usb 2-1.3: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[ .610414] usb 2-1.3: Product: FT232R USB UART
[ .610417] usb 2-1.3: Manufacturer: FTDI
[ .610421] usb 2-1.3: SerialNumber: A6008KNB
[ .613285] ftdi_sio 2-1.3:1.0: FTDI USB Serial Device converter detected
[ .613358] usb 2-1.3: Detected FT232RL
[ .613875] usb 2-1.3: FTDI USB Serial Device converter now attached to ttyUSB0
```

In my case, it is "/dev/ttyUSB0".

We will use 115200/8N1 without Hardware Flow Control.

```
$ sudo minicom -s
Serial port setup
A
```

```
/dev/ttyUSB0
F to disable the HW Flow Control
<Enter>
Save setup as..
ttyUSB0
Exit
CTRL-A Z and W
```

The last command is to enable the line wrap.

When you execute again, use "ttyUSB0" that we saved.

```
$ sudo minicom ttyUSB0
```

2.5 Update Bootloader

I assume you are using R-CAR H3eSK WS3.0.

Linux Boot is to download U-Boot with Trusted Board Boot Requirement, on other word, secured boot.

	Description
BL1	Trusted ROM (Bootloader stage 1) BL1 is the very first code that executes on the CPU after a power-on reset. It is typically burned into the Read-Only Memory (ROM) of the SoC during manufacturing. Primary Job: Initialize the platform and verify the next stage (BL2). Key Tasks: - Sets up the basic architectural state (e.g., Exception Level 3). - Initializes a small amount of internal SRAM for the next stage. - Locates and loads the BL2 image from non-volatile storage (like eMMC or SD card). - Security: It uses a public key stored in the SoC to verify the digital signature of BL2. If the signature is invalid, the boot process stops to prevent unauthorized code from running.
BL2	Trusted Boot Firmware (Bootloader Stage 2) BL2 is usually the first stage of "pluggable" firmware, meaning it resides in flash memory rather than ROM. It runs in Secure EL1. Primary Job: Initialize main memory (DDR) and act as the "dispatcher" for all subsequent stages. Key Tasks: - DRAM Initialization: This is the most critical hardware task, as later stages are often too large for internal SRAM. - Loading Multiple Images: BL2 is responsible for loading BL31, BL32 (Secure OS like OP-TEE, if used), and BL33 (Non-secure bootloader like U-Boot). - Verification: It verifies the signatures of all BL3x images before handing over control.
BL3	EL3 Runtime Software BL31 is a unique and permanent resident in the system. While BL1 and BL2 "disappear" once they finish their jobs, BL31 stays in memory while the operating system is running. Primary Job: Handle the transitions between the "Secure World" and the "Normal World." Key Tasks: - SMC (Secure Monitor Call) Handling: It acts as the gatekeeper. When the OS (Non-secure) needs to perform a secure operation—like turning on a CPU core or accessing a secure variable—it calls BL31. - Power Management: It often implements the PSCI (Power State Coordination Interface), which the Linux kernel uses to manage CPU idle states and hotplugging. - Context Switching: It saves and restores CPU registers when switching the processor between Secure

	and Non-secure states.
--	------------------------

On R-CAR H3SK, it downloads 6 files.

fname & location	Description
bootparam_sa0.srec 0x000000	Boot Parameters This is not executable code, but a data structure located at the very beginning of the flash memory (Sector 0). Role: The R-Car internal Boot ROM reads this file first. Content: It contains metadata about the flash layout, including the load addresses and sizes for the subsequent boot stages (like BL2). It tells the hardware "where to find the code."
bl2-h3ulcb.srec 0x040000	Trusted Boot Firmware This corresponds to the BL2 stage we discussed earlier. While the file name says h3ulcb, it is commonly used for the H3SK as they share a similar architecture (ULCB vs. SK). Role: This is the first "pluggable" code loaded into internal SRAM by the Boot ROM. Key Tasks: - Initializes the LPDDR4 RAM. Without this, the system cannot load larger files like U-Boot or Linux. - Initializes the system clocks and basic hardware interfaces (I2C, GPIO). - Loads and verifies the next stages (BL31 and U-Boot).
cert_header_sa6.srec 0x180000	Certificate Headers In a secure boot configuration, you will see several "cert" or "header" files. Role: These contain the digital signatures and public key information required to verify the integrity of the BL2 and BL3x images. Even if you aren't using "Secure Boot" in a development environment, the R-Car bootloader often expects these headers to be present in the flash layout to identify the image boundaries.
bl31-h3ulcb.srec 0x1C0000	EL3 Runtime Software This is the ARM Trusted Firmware (TF-A) Stage 3-1. Role: It manages the Secure Monitor. It stays resident in memory to handle power management (PSCI) and switches between the Secure World (where OP-TEE lives) and the Normal World (where Linux lives).
tee-h3ulcb.srec 0x 200000	OP-TEE OS <i>(Optional, but often present in the IPL suite)</i> Role: This is the binary for the OP-TEE operating system. If your system requires a Trusted Execution Environment, BL2 will load this into a protected region of memory.
u-boot-elf.srec 0x640000	The Bootloader / BL33 This is the final stage of the firmware before the Linux kernel. Role: U-Boot provides the user interface (the command prompt over UART) and the logic to load the Linux kernel from an SD card, eMMC, or over a network (TFTP). Format: The .srec (Motorola S-record) format is used here specifically so that the Renesas Flash Writer tool can easily parse the memory addresses and write the data to the correct location in the serial flash.

2.6 Update U-Boot parameters

Target	Parameters
NFS	<pre>=> setenv bootargs_nfs 'rw root=/dev/nfs rootwait nfsroot=192.168.1.2:/srv/nfs/50900-h3-weston,vers=3 ip=dhcp' => setenv bootcmd_nfs 'setenv bootargs \${bootargs_nfs};bootp 0x48080000 Image;tftp'</pre>

	<pre>0x48000000 \${dtb};run booti_cmd' => setenv booti_cmd 'booti 0x48080000 - 0x48000000' => setenv dtb 'r8a779m1-ulcb.dtb' => setenv ethact 'ravb' => setenv ethaddr '2e:09:0A:01:99:53' => setenv serverip '192.168.1.2' => setenv ipaddr '192.168.1.7' => saveenv => run bootcmd_nfs</pre>
SD	<pre>=> setenv bootargs_sd0 'rw root=/dev/mmcblk1p1 rootfstype=ext4 rootwait' => setenv bootcmd 'run bootcmd_sd0' => setenv bootcmd_sd0 'setenv bootargs \${bootargs_sd0};ext2load mmc 0:1 0x48080000 /boot/Image;ext2load mmc 0:1 0x48000000 /boot/\${dtb};run booti_cmd' => saveenv => run bootcmd</pre>
VBox NFS	<pre>=> setenv bootargs_vb 'rw root=/dev/nfs rootwait nfsroot=192.168.56.15:/srv/nfs/50900-h3-weston,vers=3 ip=dhcp' => bootcmd_vb 'setenv bootargs \${bootargs_vb};bootp 0x48080000 Image;tftp 0x48000000 \${dtb};run booti_cmd' => saveenv => run bootcmd_vb</pre>

2.7 Populate SDK and Install

Build SDK

```
$ export WORK=~/rcar/50900/h3-weston
$ source $WORK/poky/oe-init-build-env
$ bitbake core-image-weston-sdk -c populate_sdk
```

Install SDK

```
$ ./tmp/deploy/sdk/poky-glibc-x86_64-core-image-weston-*.sh
Select /opt/poky/3.1.11-h3-weston for SDK location.
```

SDK environment

Open new Terminal and then execute below command.

```
$ export WORK=~/rcar/50900/h3-weston
$ export TFTP_LOC=/srv/tftp/50900-h3-weston
$ export NFS_LOC=/srv/nfs/50900-h3-weston
$ export SDK_LOC=/opt/poky/3.1.11-h3-weston
$ export KERNEL_LOC=$WORK/build/tmp/work-shared/h3ulcb/kernel-source
$ export IMAGE_LOC=$WORK/build/tmp/deploy/images/h3ulcb
$ cd $WORK
$ source $SDK_LOC/environment-setup-aarch64-poky-linux
```

Note: For SDK, we did not execute “source \$WORK/poky/oe-init-build-env” command.

To validate, check \$CC and \$CROSS_COMPILE

```
$ echo $CC
aarch64-poky-linux-gcc -march=armv8-a -mtune=cortex-a57.cortex-a53 -fstackprotector-
strong -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=formatsecurity
--sysroot=/opt/poky/3.1.11-h3-weston/sysroots/aarch64-poky-linux
$ echo $CROSS_COMPILE
aarch64-poky-linux-
```

To validate SDK environment, prepare HelloWorld.c file.

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Hello World\n");
}
```

Compile and copy to NFS.

```
$ ${CC} -o HelloWorld HelloWorld.c
$ sudo cp HelloWorld $NFS_LOC/home/root
```

On Target

```
# ls
HelloWorld
# ./HelloWorld
Hello World
#
```

2.8 Prepare SD memory

At first, we have to find out memory device name. I am using USB SD memory interface dongle. If you are using SDHC card on PCMCIA, the name will be `"mmcblk0"`.

```
$ sudo fdisk -l
```

Insert USB SD memory

```
$ sudo fdisk -l
```

Compare the previous message, and find new disk, and confirm the size.

In my case, new disk has below message.

```
Disk /dev/sdd: 7.2 GiB, 7746879488 bytes, 15130624 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xa3b4b440
```

```
Device Boot Start End Sectors Size Id Type
/dev/sdd1 2048 15130623 15128576 7.2G 83 Linux
```

In my case, I found SD memory on `/dev/sdd1`.

If your SD already has a file system, Ubuntu host will automatically mount SD on `/media/$USER`. You have to unmount SD memory before you execute the below "fdisk" command. If you have "h3ulcb" mount point, use this name when you execute unmount command.

```
$ ls /media/$USER
h3ulcb
$ umount /media/$USER/h3ulcb
```

On this step, we will make a partition and prepare FileSystem and then assign the label.

R-CAR using 1 partition and using ext3 or ext4. Below example will use ext4. Please use correct memory name you found on the previous step. It will clear all data on SD memory (or your Host HDD if you typed wrong device name).

```
$ sudo fdisk /dev/sdd
Welcome to fdisk (util-linux 2.27.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Command (m for help): d
Selected partition 1
Partition 1 has been deleted. <Note: Repeat until you delete all partitions>

Command (m for help): n
Partition type
  p primary (0 primary, 0 extended, 4 free)
  e extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-15130623, default 2048): <Note: just type enter key.>
Last sector, +sectors or +size{K,M,G,T,P} (2048-15130623, default 15130623):
                                                    <Note: just type enter key.>
Created a new partition 1 of type 'Linux' and of size 7.2 GiB.

Command (m for help): t
Selected partition 1
Partition type (type L to list all types): 83
Changed type of partition 'Linux' to 'Linux'.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.

$ sudo mkfs.ext4 /dev/sdd1
mke2fs 1.42.13 (17-May-2015)
/dev/sdd1 contains a ext3 file system labelled 'h3ulcb'
  last mounted on / on Mon Sep 27 17:56:43 2021
Proceed anyway? (y,n) y
Creating filesystem with 1891072 4k blocks and 473280 inodes
Filesystem UUID: e5c8d27e-ecc9-4dcf-90ef-6fefc64b9b3d
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632
Allocating group tables: done
Writing inode tables: done
Creating journal (32768 blocks): done
Writing superblocks and filesystem accounting information: done

$ sudo e2label /dev/sdd1 h3ulcb9974
```

Now, new disk label will be "h3ulcb9974".

Install FileSystem on your SD memory

```
$ sudo mount /dev/sdd1 /mnt
$ sudo tar -xjf "$IMAGE_LOC/core-image-weston-h3ulcb.tar.bz2" -C /mnt
$ sync
```

Update Kernel Image and device tree

```
$ sudo rm /mnt/boot/*  
$ sudo cp $IMAGE_LOC/Image /mnt/boot  
$ sudo cp $IMAGE_LOC/r8a779m1-ulcb.dtb /mnt/boot  
$ sudo cp $IMAGE_LOC/r8a77951-ulcb.dtb /mnt/boot  
  
$ cd $WORK/build  
$ sudo umount /mnt
```

All done.

Target Device Deployment.

- Power off the H3SK board.
- Eject the SD card from your Host PC and insert it into the SDHI slot on the H3SK board.
- Power on the H3SK board.
- Wait for 3 seconds without pressing any keys to allow the system to bypass the U-Boot prompt and proceed with the automatic boot sequence.

3 Yocto Basic

Source directory

```
$WORK
+- build/
| +- conf/
| | +- local.conf
| | +- bblayers.conf
| +- tmp/
| | +- deploy/
| | +- log/
| | +- work/
| | +- work-shared/
+- meta-openembedded/
| +- meta-oe/
+- meta-renesas/
+- meta-vspauto/
+- poky/
  +- bitbake/
```

	Description
build/conf/local.conf	This configuration file contains the local user configurations for the build environment.
build/conf/bblayers.conf	This configuration file defines layers. It uses BBLAYERS variables to list the layers BitBake tries to find.
build/tmp	Uses this directory for all the build system's output. The TMPDIR variable points to this directory.
build/tmp/deploy	It contains any “end result” output. The DEPLOY_DIR variable points to this directory.
build/tmp/deploy/images/	It populated with the basic output objects of the build, including boot loader, kernel, root filesystem.
build/tmp/deploy/sdk	It to hold toolchain installer scripts which, when executed, install the sysroot that matches your target hardware.
build/tmp/log/	This directory contains general logs that are not otherwise placed using the package's WORKDIR .
build/tmp/work	This directory contains architecture-specific work sub-directories for packages built by BitBake.
build/tmp/work-shared/	For efficiency, the OpenEmbedded build system creates and uses this directory to hold recipes that share a work directory with other recipes. For example GCC and Kernel.

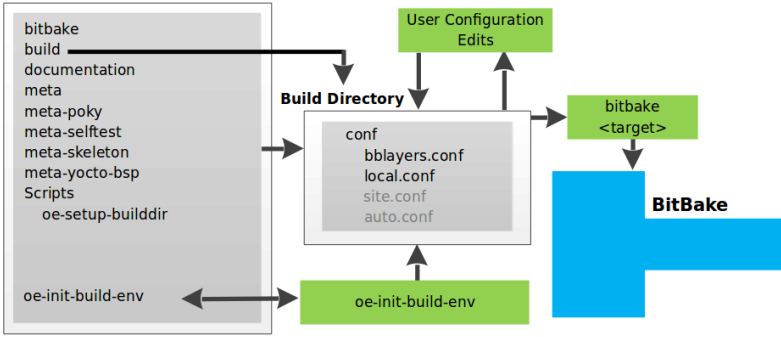
It will explain about layer, recipes, tasks, package on the target system.

3.1 User Configuration

BitBake needs some basic configuration files in order to complete a build. These files are “*.conf” files. The Build Directory has a conf directory that contains default versions of local.conf and bblayers.conf configuration files. These default configuration files are created only if they do not already exist in the Build Directory.

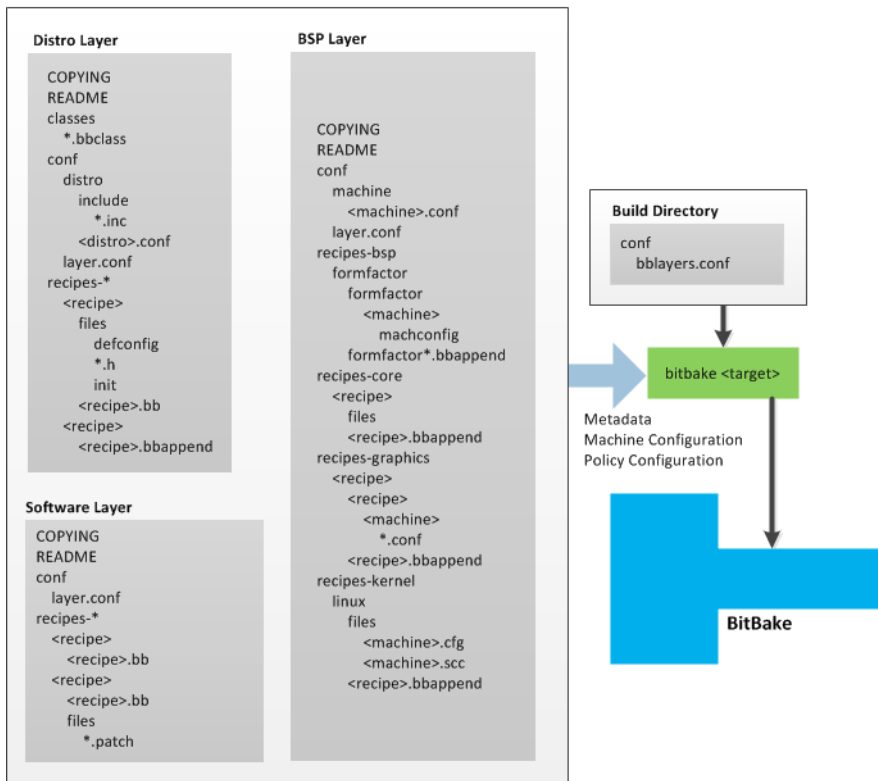
Configuration files provide many basic variables that define a build environment.

Source Directory (e.g. poky directory)



3.2 Layer

Layers



Layer	Description
Distro	A distribution layer provides policy configurations for your distribution.
BSP	A BSP layer provides machine configurations that target specific hardware. Everything in this layer is specific to the machine for which you are building the image or the SDK.
Software	A software layer provides the Metadata (.bb + Patches) for additional software packages used during the build.

BitBake uses the **conf/bblayers.conf** file, which is part of the user configuration, to find what layers it should be using as part of the build.

R-CAR H3SK is composed with below layers. On the top, I provides my new layer; meta-vspauto.

```
$ bitbake-layers show-layers
```

layer	path	priority
=====	=====	=====
meta	\$WORK/build/../../poky/meta	5
meta-poky	\$WORK/build/../../poky/meta-poky	5
meta-yocto-bsp	\$WORK/build/../../poky/meta-yocto-bsp	5
meta-rcar-gen3	\$WORK/build/../../meta-renesas/meta-rcar-gen3	6
meta-python	\$WORK/build/../../meta-openembedded/meta-python	7
meta-oe	\$WORK/build/../../meta-openembedded/meta-oe	6
meta-vspauto	\$WORK/build/../../meta-vspauto	7

Layer	Description
poky/ meta meta-poky meta-yocto-bsp	Poky is a <i>reference distribution</i> of the Yocto Project
meta-openembedded/ meta-python meta-oe	framework for embedded Linux. Include Kernel & Library base code.
meta-renesas/meta-rcar-gen3	Renesas R-CAR layer
meta-vspauto	Renesas VSPAuto team layer

3.3 recipes

\$WORK/poky/meta/recipes.txt

recipes-	Description
bsp	Anything with links to specific hardware or hardware configuration information
core	What's needed to build a basic working Linux image including commonly used dependencies
extended	Applications which whilst not essential add features compared to the alternatives in core. May be needed for full tool functionality.
kernel	The kernel and generic applications/libraries with strong kernel
multimedia	Codecs and support utilities for audio, images and video
support	Recipes used by other recipes but that are not directly included in images

```
$ bitbake-layers show-recipes
```

=== Available recipes: ===	Layer	PV
core-image-weston:	meta	1.0
gststreamer1.0-plugins-base:	meta	1.16.3
kernel-module-vspmfif:	meta-rcar-gen3	1.0
linux-renesas:	meta-rcar-gen3	5.10.41+gitAUTOINC+4846ef7755
v4l-utils:	meta-oe	1.18.1
weston:	meta	8.0.0
..	..	..

\$ bitbake-layers show-append

3.4 tasks

Command, for example linux-renesas

```
$ bitbake "recipe_name" -c listtasks
```

task	Descriptions
do_fetch	Fetch – get the source code
do_unpack	Extract – unpack the sources
do_prepare_recipe_sysroot	Sets up an isolated, recipe-specific environment containing all the dependent libraries and headers needed to compile a particular software package.
do_patch	Patch – apply patches for bug fixes and new capability
do_configure	Configure – set up your environment specifications
do_compile	Build – compile and link
do_install	Install – copy files to target directories
do_package	Analyzes the content of the holding area and splits it into subsets based on available packages and files
do_packagedata	Creates package metadata used by the build system to generate the final packages
do_populate_sysroot	Copies a subset of files installed by do_install into the sysroot in order to make them available to other recipes
do_package_qa	Runs QA checks on packaged files
do_package_write_rpm	Creates the actual RPM packages and places them in the Package Feed area
do_package_XXX	do_package_qa_setscene - Runs QA checks on packaged files (sets scene version) do_package_setscene - Analyzes the content of the holding area and splits it into subsets based on available packages and files (sets scene version) do_package_write_rpm_setscene - Creates the actual RPM packages and places them in the Package Feed area (sets scene version) do_packagedata_setscene - Creates package metadata used by the build system to generate the final packages (sets scene version)
	Other tasks you should know.
do_cleanall	Clean all, includes cache, downloaded files. You can start from beginning
do_clean	Clean all output files for a target.
do_menuconfig	Runs 'make menuconfig' for the kernel
do_compile_kernelmodules	Compiles loadable modules for the Linux Kernel
do_update_vspauto_dtbs	appended from vspauto meta layer

For linux-renesas

```
$ bitbake linux-renesas -c cleanall
$ bitbake linux-renesas -c -f menuconfig
$ bitbake linux-renesas -c -f compile
$ bitbake linux-renesas -c -f deploy
```

Note: "-f" is for the forced run that keeps the modified source code.

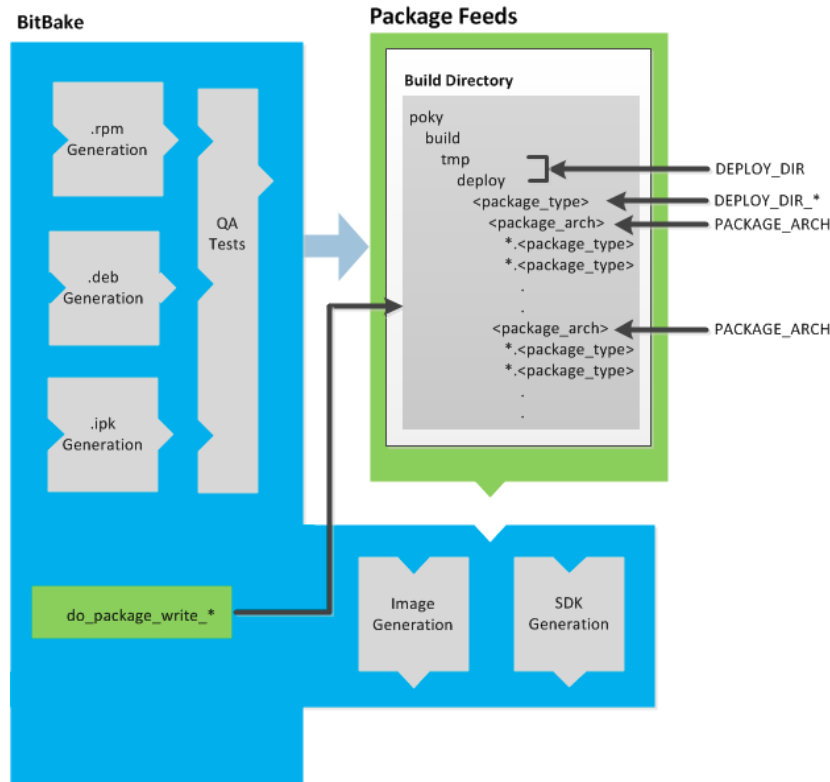
And execute to prepare all packages

```
$ bitbake core-image-weston
```

we can see the task_order on \$WORKDIR/temp/log.task_order.

3.5 packages

Each recipes generates the packages, and it is used when we generates the image.



3.6 Bitbake command options

Option	Meaning
-c <task>	execute <task> for the image or recipe
-f	Force execution of the operation. If you modified the base code, use “-f -c compile”
-v	show verbose output
-DDD	show lots of debug information
-e	output recipe environment variables ex: bitbake <recipe> -e grep ^FILES=
-c listtasks <recipe>	show the tasks associated with an image or individual recipe

Examples

commands	Description
bitbake <image> -g cat pn-buildlist	How to get the Image package list

bitbake <module> -c cleanall	to clean a module
bitbake <image> -c cleanall	to clean an image
bitbake <image> -c fetchall	to load the packages to a local hard drive
bitbake <image> -c listtasks	to get the list of tasks
bitbake-layers show-append	to display all appends
bitbake <recipe-or-image-name> -g -u depexp	to display all layers
bitbake <kernel_module> -c menuconfig	to configure the kernel
bitbake <image> -c populate_sdk	to make an SDK image

3.7 .bb and .bbappend

“.bb” is a recipes extension. If you need to add, you can append file that has “.bbappend” extension. The syntax is similar Makefile system.

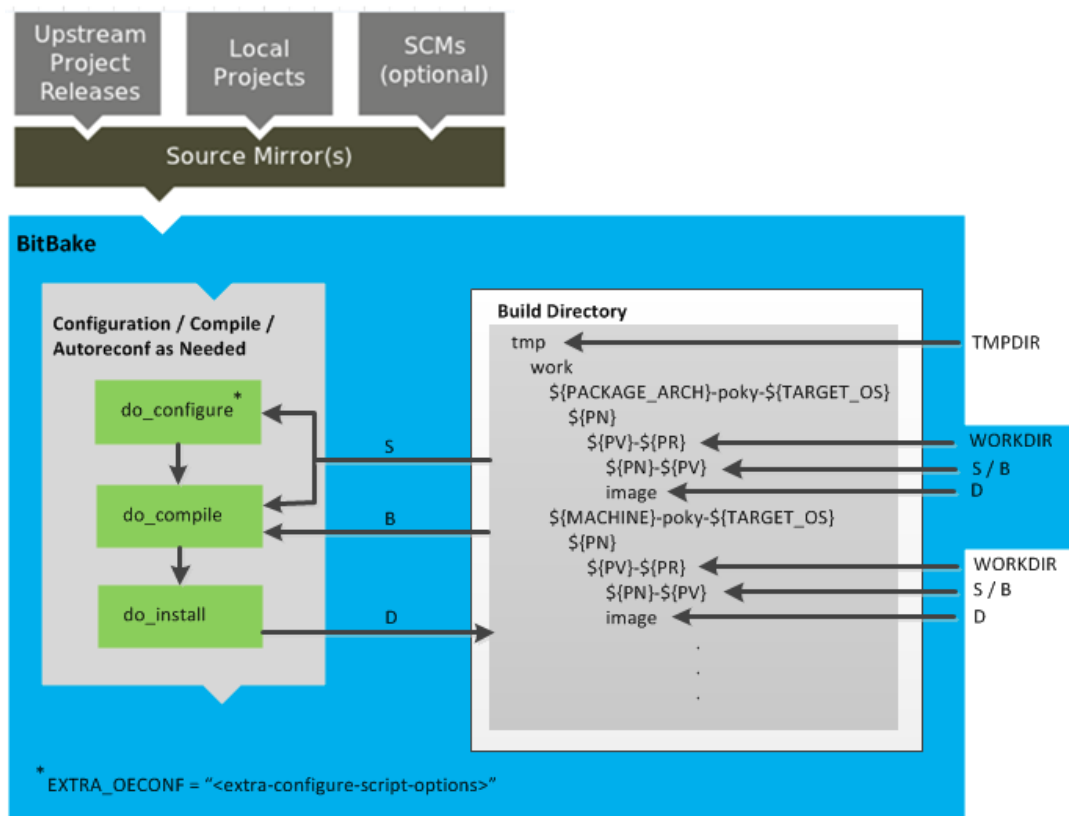
Assign syntax	Meaning
VAR = "foo"	simple assignment
VAR ?= "foo"	assign if no other value is already assigned (default assignment)
VAR ??=foo	weak default assignment takes lower precedence than ?=
VAR = "stuff \${OTHER_VAR}"	variable expansion. OTHER_VAR expanded at time of reference to VAR
VAR := "stuff \${OTHER_VAR}"	immediate variable expansion OTHER_VAR expanded at time of parsing this line
VAR += "foo"	append with space
VAR .= "foo"	append without space
OVERRIDES="string1:string2" VAR = "foo" VAR_string1 = "bar"	alternate/override value if string1 is listed in OVERRIDES, use "bar" for value of VAR, otherwise use "foo"
OVERRIDES="string1:string2" VAR = "foo" VAR_append_string1 = "bar"	conditional append if string1 is in OVERRIDES, then append "bar" to the value of VAR
BBVERSIONS="1.0 1.8 string" VAR="foo" VAR_string="bar"	range-specific conditional If the version of the package is in the specified range (1.0-1.8 in this example), then perform an override on the indicated variable
VAR = "foo \${@<line-of-python-code>}"	python code expansion ex: VAR = "the date is: \${@time.strftime('%Y%m%d',time.gmtime())}"
VAR[some_flag]="foo"	associate a subsidiary flag value to a variable a few subsidiary flag value names are well-defined: "dirs", "cleandirs", "noexec", "nostamp", "fakeroot", "umask", "deptask", "rdeptask", "recdeptask", "recrdeptask" Flag values appear to be used exclusively with task definitions (i.e. do_sometask)

Syntax	Meaning
include foo	include file named "foo", search on BBPATH
require [<path>] foo	include file named "foo", failing if not found exactly where specified
inherit foo	include definitions from foo.bbclass
do_sometask() { <shell code>	define a task using shell code

} python do_sometask { <python code> }	define a task using python code
addtask sometask (before after) other_task	adds a defined task to the list of tasks, with the ordering specified. Zero or more 'before' or 'after' clauses can be used.
inherit externalsrc EXTERNALSRC = "/some/path" # EXTERNALSRC_BUILD = "/some/path"	Fetch source from /some/path You can place this at the bottom of a .bb or .bbappend file to override where the source will be fetched from. This is very convenient if you are currently working on the source code. Depending on the type of build (eg, 'inherit module' for out of tree Linux kernel modules) you may or may not need to set EXTERNALSRC_BUILD.

Recipe environment variables

variable	Description
TOPDIR	Points to the build directory. BitBake automatically sets this variable. The default TOPDIR is \$WORK/build
THISDIR	Location of .bb
WORKDIR	Normally, \$WORK/build/tmp/work/aarch64-poky-linux/\$PN/1.0-r0
S	Fetches Source location. S = \$WORKDIR S=\$WORKDIR/zip_dir_name if SRC_URI has zipped directory,
B	Build location B = \$WORKDIR
D	Destination location D = \$WORKDIR/image
T	Points to a directory where BitBake places temporary files, which consist mostly of task logs and scripts, when building a particular recipe. T = \$WORKDIR/temp
PN	recipe name. \${BPN}:the base recipe name without any special suffix or version numbers.
PV	Version of recipes. Default is "1.0".
PR	Revision of the recipe. Default is "r0"
FILE	the absolute path to the ".bb" or ".bbappend" file.
FILESPATH	For batbake search path for "file://" URI.
FILESEXTRAPATHS	Ex: FILESEXTRAPATHS_prepend := "\${THISDIR}/src:"
DEPENDS	Lists a recipe's build-time dependencies. See RDEPENDS also.
RDEPENDS	Lists a package's runtime dependencies that must be installed in order for the built package to run correctly. If a package in this list cannot be found during the build, you will get a build error.
PROVIDES	A list of package name aliases that a package also provides. These aliases are useful for satisfying runtime dependencies of other packages both during the build and on the target (as specified by RDEPENDS).
RPROVIDES	A list of package name aliases that a package also provides. These aliases are useful for satisfying runtime dependencies of other packages both during the build and on the target (as specified by RDEPENDS).
SRC_URI	The list of source files — local or remote. file://: Fetches files, which are usually files shipped with the Metadata, from the local machine. The path is relative to the FILESPATH variable. files: files within a directory, which is named files and is also alongside the recipe or append file. git://: Fetches files from a Git revision control repository.



To see the recipe environment variables, use "-e" command. For example,

```
$ bitbake linux-renesas -e | grep ^FILES=
```

Example

```
PN = "${@bb.parse.vars_from_file(d.getVar('FILE', False),d)[0] or 'defaultpkgname'}"
TMPDIR = "${TOPDIR}/tmp"
CACHE = "${TMPDIR}/cache"
STAMP = "${TMPDIR}/${PN}/stamps"
T = "${TMPDIR}/${PN}/work"
B = "${TMPDIR}/${PN}"
```

To debug variable, use below examples

```
do_compile() {
    echo "CFLAGS=" ${CFLAGS}
    echo "LDFLAGS=" ${LDFLAGS}
    ...
}

python do_display_banner() {
    bb.plain("*****");
    bb.plain("*");
    bb.plain("* Example recipe created by bitbake-layers *");
    bb.plain("*");
    bb.plain("*****");
}

addtask display_banner before do_build
```

3.8 Log

When developing with the Yocto Project, understanding where logs are generated is crucial for debugging build failures, tracing configuration issues, and optimizing recipe execution. BitBake categorizes its logs into three main layers: the global daemon level, the session level, and the specific task level.

location	Description
<code>\$WORK/build/bitbake-cookerdaemon.log</code>	The Global Server Log This log tracks the background BitBake server process (Cooker daemon). It records server initialization, internal communication, memory usage, and critical system errors that happen outside of individual task execution.
<code>\$WORK/build/tmp/log/cooker/h3ulcb</code>	The Session Console Log Contains history logs of your BitBake command execution. The most notable file here is <code>console-latest.log</code>, which captures the exact terminal output (including the ASCII progress bars, parsing status, and summary errors) of your most recent bitbake run.
<code>\$WORKDIR/temp</code>	The Recipe Task Log Sandbox The primary debugging directory for a specific recipe. It contains detailed runtime files for every single task executed during the recipe build lifecycle. If you want to see the task order, see <code>log.task_order</code>. For each task, see <code>log.do_XXX</code> and <code>run.do_XXX</code> files.

4 Bitbake steps

4.1 Source Fetching

The first stages of building a recipe are to fetch and unpack the source code

To do_fetch and do_unpack, it using SRC_URI on .bb file, and download at \${WORKDIR}. If SRC_URI file was zipped, it unzip on \${WORKDIR}/filename.

```
tmp
work
  ${PACKAGE_ARCH}-poky-{TARGET_OS}    --->TMPDIR
  ${PN}
  ${PV}                                ---->WORKDIR
  sources                              ----> UNPACKDIR
  ${BP}                                ----> S
${MACHINE}-poky-{TARGET_OS}
${PN}
${PV}
sources
${BP}
```

On do_patch step, if you used the “.patch” file, it download the patch file on \${WORKDIR}, and do patch on \${BP} location.

On do_prepare_recipe_sysroot, it generates recipe-sysroot that the subsequent tasks of the recipe (notably do_configure and do_compile) can access the libraries, headers, and similar files built by the recipes

```
tmp
work
  ${PACKAGE_ARCH}-poky-{TARGET_OS}    --->TMPDIR
  ${PN}
  ${PV}                                ---->WORKDIR
  sources                              ----> UNPACKDIR
  ${BP}                                ----> S
  recipe-sysroot
  recipe-sysroot-native
${MACHINE}-poky-{TARGET_OS}
${PN}
${PV}
sources
  ${BP}
recipe-sysroot
recipe-sysroot-native
```

4.2 Configure, Compile and Staging (Install)

On do_configure, it enabling and disabling any build-time and configuration options for the software being built. Compilation occurs in the directory pointed to by the **B** variable. Realize that the **B** directory is, by default, the same as the **S** directory.

After `do_compile` task, `do_install` task copies files from the B directory and places them in a holding area pointed to by the D variable. Packaging occurs later using files from this holding directory.

```
tmp
work
  ${PACKAGE_ARCH}-poky-{TARGET_OS}    --->TMPDIR
    ${PN}
      ${PV}                            ---->WORKDIR
        sources                        ----> UNPACKDIR
          ${BP}                        ----> S / B
            image                      ----> D
              ${PN}
            recipe-sysroot
            recipe-sysroot-native
  ${MACHINE}-poky-{TARGET_OS}
    ${PN}
      ${PV}
        sources
          ${BP}
            image
              ${PN}
            recipe-sysroot
            recipe-sysroot-native
```

4.3 Package Splitting

After `do_install` staging task, the build system analyzes the result and split the output into packages.

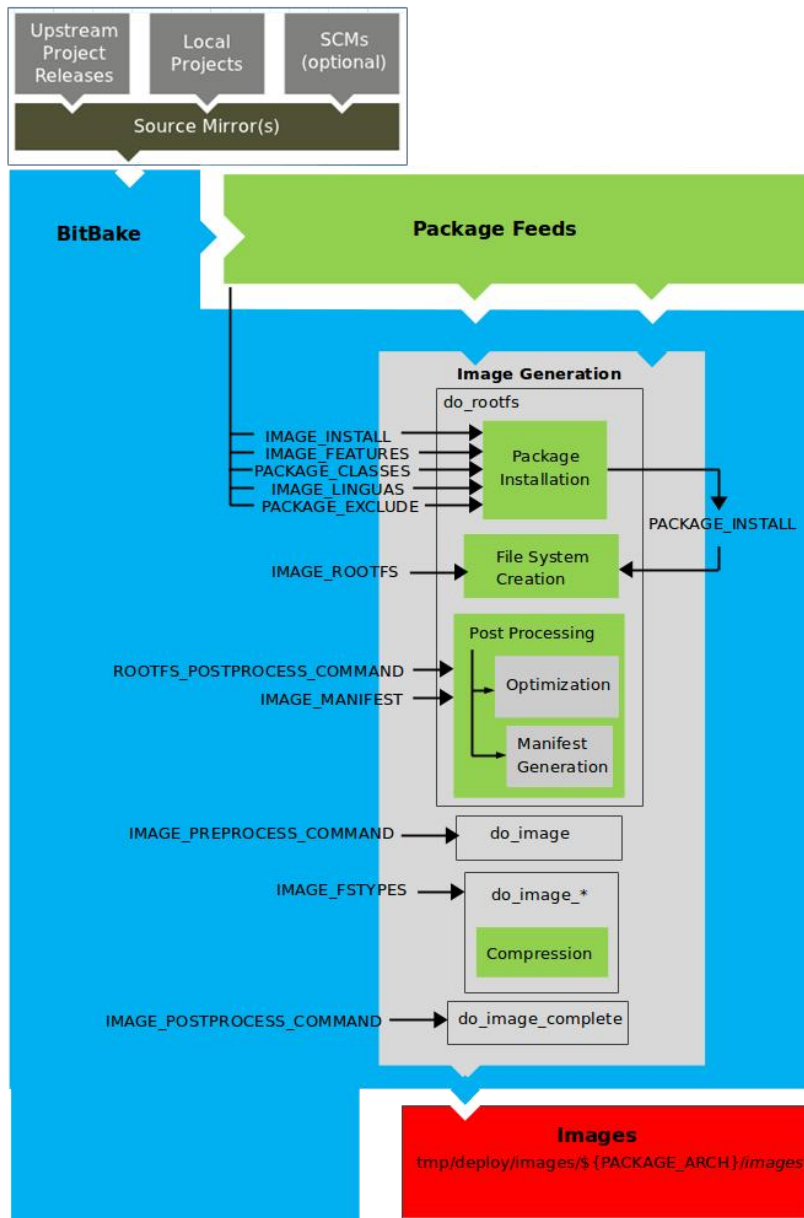
```
tmp
work
  ${PACKAGE_ARCH}-poky-{TARGET_OS}    --->TMPDIR
    ${PN}
      ${PV}                            ---->WORKDIR
        sources                        ----> UNPACKDIR
          ${BP}                        ----> S / B
            image                      ----> D
              ${PN}
            package                    ---->PKGDEST
            pkgdata                    ---->PKGDESTWORK
            packages-split            ---->PKGDEST
              ${PN}
            recipe-sysroot
            recipe-sysroot-native
  ${MACHINE}-poky-{TARGET_OS}
    ${PN}
      ${PV}
        sources
          ${BP}
            image
              ${PN}
            package                    ---->PKGDEST
            pkgdata                    ---->PKGDESTWORK
            packages-split            ---->PKGDEST
```

```
${PN}  
recipe-sysroot  
recipe-sysroot-native
```

do_package, do_packagedata and do_populate_sysroot tasks are executed.

4.4 Image Generation

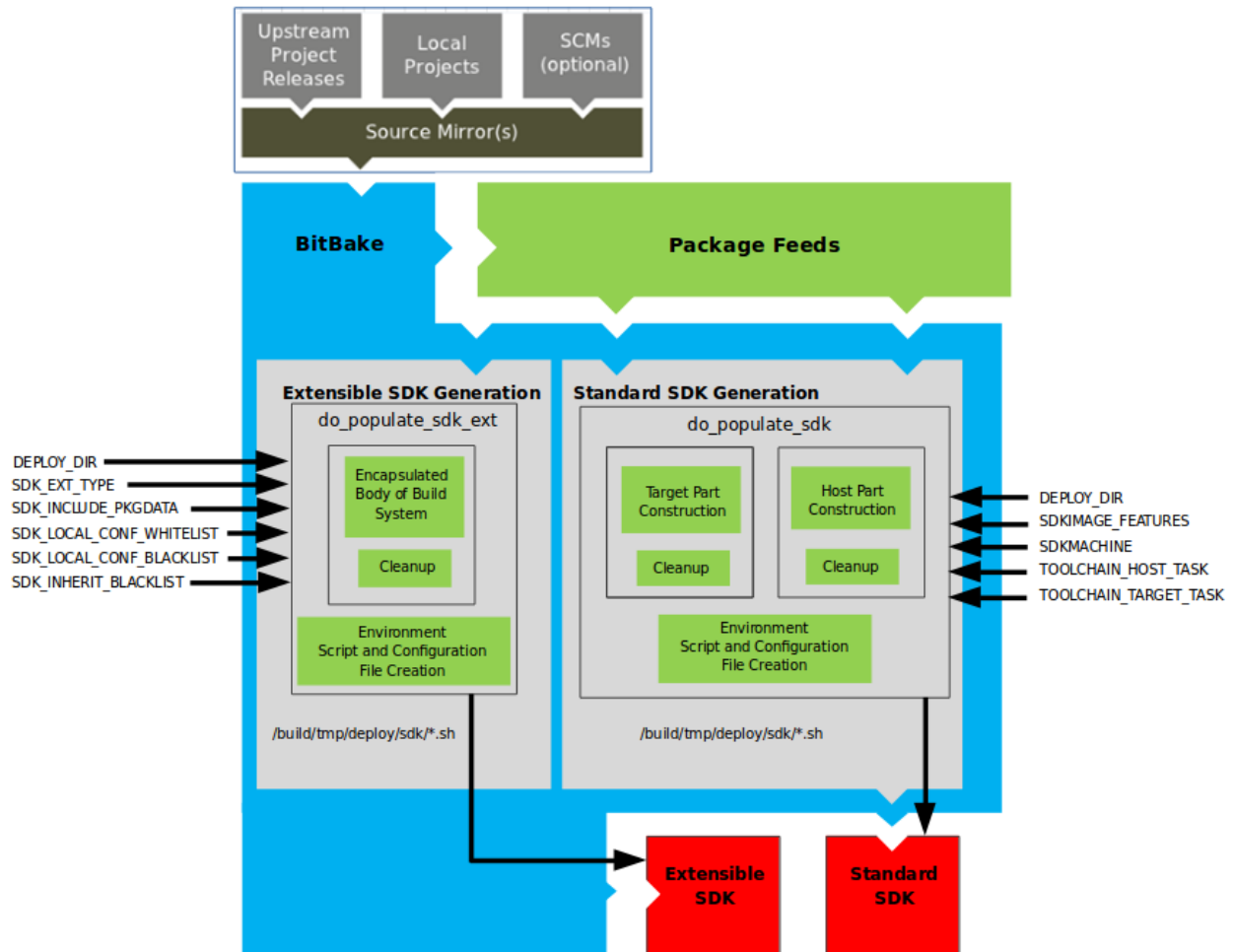
Once packages are split and stored in the Package Feeds area, the build system uses BitBake to generate the root filesystem image:



The image generation process consists of several stages and depends on several tasks and variables. The `do_rootfs` task creates the root filesystem (file and directory structure) for an image.

4.5 SDK Generation

Like image generation, the SDK script process consists of several stages and depends on many variables. The `do_populate_sdk` and `do_populate_sdk_ext` tasks use these key variables to help create the list of packages to actually install.



4.6 R-CAR Board Support Package

Below is R-CAR BSP Layers

```
.
+- build/
| +- conf/
|   +- bblayers.conf
|   +- local.conf
+- meta-openembedded/
```

```

| +- meta-oe/
| +- meta-python/
| +- ...
+- meta-renesas/
| +- meta-rcar-adas/
| +- meta-rcar-bsp/
| +- meta-rcar-gen3/
| | +- conf/machine/           <Hardware Configuration Options>
| | +- docs/sample/conf/
| | | +- h3ulcb/
| | | +- m3ulcb/
| | | +- salvator-x/
| | | +- ...
| | +- recipes-bsp/
| | +- recipes-graphics/      <Display, GPU support>
| | +- recipes-kernel/        <Linux Kernel>
| | +- recipes-multimedia/
| | +- ...
| +- ...
+- meta-vspauto/
+- poky/
+- oe-init-build-env

```

It can supports 3 SoC BSPs (h3ulcb, m3ulcb and salvator-x), and 3 different meta layers.

As you can see on current "conf/bblayers.conf" we are using meta-rcar-gen3.

```

BBLAYERS ? = " \
    ${TOPDIR}/../poky/meta \
    ${TOPDIR}/../poky/meta-poky \
    ${TOPDIR}/../meta-renesas/meta-rcar-gen3 \
    ${TOPDIR}/../meta-openembedded/meta-python \
    ${TOPDIR}/../meta-openembedded/meta-oe \
    "
BBLAYERS += " ${TOPDIR}/../meta-vspauto "

```

5 Modify Linux Kernel

5.1 Traditional Method

```
$ export ARCH=arm
$ export CROSS_COMPILE=arm-linux-gnueabihf-
$ cd <your Linux Kernel location>
$ make cleanall
$ cp ./arch/arm64/configs/defconfig .config <or use make $(target)_defconfig command>
$ make menuconfig
$ make -j$(nproc) zImage
$ make modules
$ make dtbs
$ make modules_install INSTALL_MOD_PATH=<...>
$ cp arch/armboot/zImage <your TFTP location>
$ cp arch/armboot/*.dtb <uour TFTP location>
```

5.2 Yocto Linux Kernel location

location	PATH
src	\$WORK/build/tmp/work-shared/h3ulcb/kernel-source
work	\$WORK/build/tmp/work-shared/h3ulcb/kernel-build-artifacts
deploy	\$WORK/build/tmp/deploy/images/h3ulcb

Before you modify Kernel, make a backup

```
$ cd $KERNEL_LOC/..
$ cp -rf kernel_src kernel_src_org
```

If you received the patch file, you can use the patch command with “-p1” option. For example

```
$ cd $KERNEL_LOC/..
$ cp ~/Downloads/kernel-patch-231116.patch
$ patch -p1 < kernel-patch-231116.patch
$ rm kernel-patch-231116.patch
```

5.3 Create new device driver

\$KERNEL_LOC/drivers/media/i2c/raa279974.c

```
/*! \file raa279974.c
 * \author Brian Kang
 *
 */
...
static const struct of_device_id raa279974_of_table[] = {
    { .compatible = “raa,raa279974”, },
    { }
};
MODULE_DEVICE_TABLE(of, raa279974_of_table);
...
```

The compatible string value will be used to link driver and device tree.

`$KERNEL_LOC/drivers/media/i2c/Kconfig`

```
config VIDEO_RAA279974
    tristate "Renesas RAA279974 AHL decoder"
    depends on VIDEO_V4L2 && I2C
    help
        V4l2 subdevice driver for Renesas RAA279974
        RAA279974 video decoder
```

`$KERNEL_LOC/drivers/media/i2c/Makefile`

```
obj-$(CONFIG_VIDEO_RAA279974) += raa279974.o
```

Enable CONFIG_VIDEO_RAA279974 on menuconfig

```
$ bitbake linux-renesas -f -c menuconfig
Device Drivers -->
  <*> Multimedia support -->
    Media ancillary drivers -->
      Video decoders -->
        <*> Renesas RAA279974 AHL decoder
<Save>
```

To confirm, open `$KERNEL_LOC/./kernel-build-artifacts/.config`, and search below string.

```
...
CONFIG_VIDEO_RAA279974=y
...
```

5.4 Create new Device Tree

On the traditional method, you have to add device tree on Kconfig and Makefile. On the Yocto, you have to add new DTS on "arch/arm64/boot/dts/renesas/", but it will be overwritten all your modified kernel source that includes your new DTS file. To add new DTS, use `$WORK/meta-renesas/meta-rcar-gen3/conf/machine/h3ulcb.conf` file. For example, add below line.

```
KERNEL_DEVICETREE += "renesas/r8a778m1-ulcb-yours.dtb"
```

But, I am using linux-renesas_5.10.bbappend on meta-vspauto layer and will explain later

```
#include "r8a779m1-ulcb.dts"

/ {
    model = "RENESAS H3ULCB-2G 9974 AHL on r8a779m1"
    compatible = "renesas,h3ulcb", "renesas,r8a779m1", "renesas,r8a7795";
    ...
};
&i2c4 {
    status = "okay";

    raa279974@3D {
        compatible = "raa,raa279974";
        reg = <0x3D>
        #address-cells = <1>;
```

```
        #size-cells = <1>;
        ah1,debug = <1>;
        ...
    };
};
...
```

The compatible string is used to link device tree and device driver.

Compile and deploy.

```
$ bitbake linux-renesas -f -c compile
$ bitbake linux-renesas -f -c deploy
```

5.5 Test new device tree and device driver

Copy to TFTP

```
$ sudo cp $IMAGE_LOC/Image $TFTP_LOC
$ sudo cp $IMAGE_LOC/r8a779m1-ulcb.dtb $TFTP_LOC
$ sudo cp $IMAGE_LOC/r8a779m1-ulcb-9974.dtb $TFTP_LOC
```

```
$ cd /srv/tftp
$ sudo rm -f Image
$ sudo ln -s 50900-h3-weston/Image Image
$ sudo rm -f r8a779m1-ulcb.dtb
$ sudo ln -s 50900-h3-weston/r8a779m1-ulcb-9974.dtb r8a779m1-ulcb.dtb
$ cd $WORK/build
```

Reboot Target machine. Type any key within 3 sec, and type below command

```
> run bootcmd_nfs
```

BootMessage

```
[ 1.684975] In raa279974_probe
[ 1.688065] raa279974 4-003d: chip find @ 3d(8bit:7a) (e66d8000.i2c)
[ 1.694962] Renesas raa279974 id:74 rev: 1 is found.
[ 1.700032] raa279974 ch0 0:1:0 ch1 0:1:0 ch2 0:1:0 ch3 0:1:0 port4 fmt:0 4ch 4lanes
[ 1.707864] random: fast init done
[ 1.747895] Connect 9971 ch0 (16|10) ID:71 Rev:04
[ 1.754669] Connect 9971 ch1 (16|11) ID:71 Rev:04
[ 1.761443] Connect 9971 ch2 (16|12) ID:71 Rev:04
[ 1.768212] Connect 9971 ch3 (16|13) ID:71 Rev:04
[ 1.790272] Connect 0X01 ch0 (36|30) SW Version 0000
[ 1.837895] Connect 0X01 ch1 (36|31) SW Version 0000
[ 1.885523] Connect 0X01 ch2 (36|32) SW Version 0000
[ 1.933151] Connect 0X01 ch3 (36|33) SW Version 0000
[ 2.085775] raa279974_check_and_set_testpattern ch1 En testpattern
[ 2.093663] raa279974_check_and_set_testpattern ch2 En testpattern
[ 2.101547] raa279974_check_and_set_testpattern ch3 En testpattern
[ 2.108484] raa279974_check_and_set_testpattern ch0 En testpattern
[ 2.115422] raa279974_check_and_set_testpattern ch0 Clean testpattern
[ 2.123492] Finish raa279974_probe
```

On device Driver, it has '#define DEBUG' on the top, and used bk_pr_debug().

```
#define DEBUG

#include <...>

#define bk_pr_err(fmt,args...) pr_err(fmt,##args)
#define bk_pr_debug(fmt,args...) pr_debug(fmt,##args)
#define bk_pr_info(fmt,args...) pr_debug(fmt,##args)
...
```

If you use "dmesg", I show more debug messages.

```
# dmesg
# dmesg -c
```

Linux Kernel Log Levels

	Name	Description
0	KERN_EMERGE	The system is unusable (e.g., imminent crash)
1	KERN_ALERT	Immediate action is required
2	KERN_CRIT	Critical conditions like hardware or software errors
3	KERN_ERR	Non-critical error conditions
4	KERN_WARNING	Potential issues that should be addressed.
5	KERN_NOTICE	Normal but significant events
6	KERN_INFO	General operation messages
7	KERN_DEBUG	Detail messages for troubleshooting

To see current log level on Target, execute below command.

```
# cat /proc/sys/kernel/printk
4 4 1 7
```

1st:current console_loglevel
2nd:default message_loglevel. It is used for messages without a specific log level.
3rd:minimum_console_loglevel. It indicates the minimum loglevel which can be used for console_level.
4th:default_console_loglevel. The default loglevel used for console_loglevel at boot time.

If you want to change current console_level to 7, use

```
# echo "7" > /proc/sys/kernel/printk
```

5.6 Kernel module

We include the rcar_imr.ko file that is on lib/modules/5.10.41-yoctostandard/kernel/drivers/media/platform.

Makefile

```
CFILE = rcar-imr.c

obj-m += rcar-imr.o

all:
    make -C $(KERNELSRC) M=$(PWD) modules
```

kernel-module-rcar-imr.bb

```
DESCRIPTION = "R-CAR IMR-LX4 Driver"
require include/rcar-gen3-modules-common.inc
LICENSE = "CLOSED"
```

```

inherit modules
FILEEXTRAPATHS_prepend := "${THISDIR}/rcar-imr-files:"
DEPENDS = "linux-renesas"
PN = "kernel-module-rcar-imr"
PR = "r0"
SRC_URL = file://Makefile file://rcar-imr.c file://rcar-imr.h
S = ${WORDDIR}
KERNEL_MODULES_PACKAGE_SUFFIX = ""

do_compile() {
    make all
}
do_install() {
    install -d ${D}/lib/modules/${KERNEL_VERSION}/extra/
    install -m 644 ${S}/rcar-imr.ko ${D}/lib/modules/${KERNEL_VERSION}/extra/
}
PACKAGES = " ${PN} ${PN}-dev "
FILES_${PN} = " /lib/modules/${KERNEL_VERSION}/extra/rcar-imr.ko "
RPROVIDES_${PN} += "kernel-module-rcar-imr"
KERNEL_MODULE_AUTOLOAD_append = " rcar-imr"

```

\$WORK/build/local.conf

```
CORE_IMAGE_EXTRA_INSTALL_append = "kernel-modules"
```

```

/
├── etc
│   ├── modprobe.d
│   │   ├── blacklist.conf
│   │   ├── cmemdrv.conf
│   │   ├── mmngr.conf
│   │   ├── mmngrbuf.conf
│   │   ├── pvrsvkm.conf
│   │   ├── rcar_imr.conf
│   │   └── uio_pdrv_genirq.conf
│   └── modules-load.d
│       ├── mmngr.conf
│       ├── mmngrbuf.conf
│       ├── rcar_imr.conf
│       ├── uio_pdrv_genirq.conf
│       ├── vsp2.conf
│       ├── vspm.conf
│       └── vspm_if.conf
├── lib/modules/5.10.41-yocto-standard/kernel/drivers/media/platform
│   └── rcar_imr.ko
└── lib/modules/5.10.41-yocto-standard/extra/
    ├── cmemdrv.ko
    ├── cryptodev.ko
    ├── mmngr.ko
    ├── mmngrbuf.ko
    ├── pvrsvkm.ko
    ├── qos.ko
    ├── uvcs_drv.ko
    ├── vsp2.ko
    └── vspm.ko

```

```
└─vspm_if.ko
```

```
core-image-weston.bbappend
```

```
IMAGE_INSTALL_append = " kernel-modules-rcar-imr kernel-modules-rcar-imr-dev"
```

```
$ bitbake core-image-weston
```

6 On SDK

Open new terminal, and execute below command.

```
export WORK=~/rcar/50900/h3-weston
export TFTP_LOC=/srv/tftp/50900-h3-weston
export NFS_LOC=/srv/nfs/50900-h3-weston
export SDK_LOC=/opt/poky/3.1.11-h3-weston
export KERNEL_LOC=$WORK/build/tmp/work-shared/h3ulcb/kernel-source
export IMAGE_LOC=$WORK/build/tmp/deploy/images/h3ulcb
cd $WORK
source $SDK_LOC/environment-setup-aarch64-poky-linux
```

Note: Do not execute "source poky/oe-init-build-env" that we did for Yocto.

To confirm, check \$CC and \$CROSS_COMPILE

```
bkang@kona:~/rcar/50900/h3-weston$ echo $CC
aarch64-poky-linux-gcc -march=armv8-a -mtune=cortex-a57.cortex-a53 -fstackprotector-
strong -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=formatsecurity
--sysroot=/opt/poky/3.1.11-h3-weston/sysroots/aarch64-poky-linux
bkang@kona:~/rcar/50900/h3-weston$ echo $CROSS_COMPILE
aarch64-poky-linux-
```

6.1 Application

To validate SDK environment, prepare HelloWorld.c file.

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Hello World\n");
}
```

Compile and copy to NFS.

```
$ ${CC} -o HelloWorld HelloWorld.c
$ sudo cp HelloWorld $NFS_LOC/home/root
```

On Target

```
# ls
HelloWorld
# ./HelloWorld
Hello World
#
```

6.2 Libraries

I have below two examples, but in this case, I will explain the simpler v4l2-utils.

```
+-- recipes-multimedia
   +- v4l2apps
     +- v4l2-utils_1.18.1.bbappend
     +- v4l2-utils
       +- vspauto-v4l2-utils-1.18.1.patch
```

```
+-- recipes-graphics
+-- wayland
+-- libgbm.bbappend
+-- Weston_8.0.0.bbappend
+-- libgbm
|   +- vspauto-libgbm-1.0.patch
+-- Weston_8.0.0
    +- vspauto-weston-8.0.0.patch
```

Let's assume, vspauto-v4l2-util-1.18.1.patch has below modifications

```
rcar/h3sk/50900/h3ulcb-weston/build/tmp/work/
+- aarch64-poly-linux/v4l-utils/
+-- 1.18.1-r0
|   +- vspauto-v4l2-util-1.18.1.patch
|   +- v4l-utils-1.18.1
|       +- include
|           +- v4l2-control.h
|           +- v4l2-subdev.h
|           +- Makefile.am      <Add v4l2-ctl-routing.cpp v4l2-ctl-mbus-config.cpp>
|           +- Makefile.in
|           +- v4l2-ctl-common.cpp
|           +- v4l2-ctl.cpp
|           +- v4l2-ctl.h
|           +- v4l2-ctl-mbus-config.cpp
|           +- v4l2-ctl-routing.cpp
+- 1.18.1-r0-base-240830
```

It updates Makefile.am, so it needs to update the automake to update the Makefile.in.

```
$ bitbake v4l0utils -f -c configure
```

And then compile and install to \$WORKDIR/image

```
$ bitbake v4l-utils -f -c compile
$ bitbake v4l-utils -f -c install
$ bitbake v4l-utils
```

If you want to go-back, use "bitbake v4l-utils -c clean".

```
rcar/h3sk/50900/h3ulcb-weston/build/tmp/work/
+- aarch64-poly-linux/v4l-utils/
+-- 1.18.1-r0/
|   +- image/
|       +- etc
|       +- lib
|           +- libmediactl.so.0.0.0
|           +- libmediactl.so.0
|           +- libmediactl.so
|   +- usr
|       +- bin
|       +- v4l2-ctl
```

Let's assume it has two files, one v4l2-ctl executable file and libmediactl.so dynamic library file.

To install executable file to NFS, use below example commands

```
$ cd $WORK/build/tmp/work/aarch64-poly-linux/v4l-utils/1.18.1-r0/image/usr/bin
```

```
$ sudo cp v4l2-ctl $NFS_LOC/usr/bin
```

To validate

```
# v4l2-ctl --help-subdev
```

To install dynamic library file to NFS, use below example commands

```
$ cd ../../lib
```

```
$ sudo cp -d libmediactl.so.* $NFS_LOC/lib
```

or copy libmediactl.so.0.0.0 and then re-link .so file

```
$ cd ../../lib
```

```
$ sudo cp -d libmediactl.so.* $NFS_LOC/lib
```

```
$ sudo -i
```

```
# cd /srv/nfs/50900-h3-weston/lib
```

```
# ls -al libmediactl.so*
```

```
# rm libmediactl.so.0.0.0
```

```
# rm libmediactl.so.0
```

```
# ln -s libmediactl.so.0.0.0 libmediactl.so.0
```

```
# ln -s libmediactl.so.0 libmediactl.so
```

```
# exit
```

```
$
```

To validate, use “ldd” command that using libmediactl.

7 New Layer

7.1 Create new layer

```
$ bitbake-layers create-layer meta-test
```

```
$WORK/meta-test
├── COPYING.MIT
├── README
├── conf
│   └── layer.conf
├── recipes-example
│   └── example
│       └── example_0.1.bb
```

And add your layer on \$WORK/build/conf/bblayers.conf

```
BBLAYERS += " ${TOPDIR}/../meta-test"
```

So far, it adds the "example" recipes, but it just prints a banner message when you execute do_build.

```
$ bitbake example -c build
*****
*                                     *
* Example recipe created by bitbake-layers *
*                                     *
*****
```

On layer.conf, add below two lines to add the search PATH. It gives you more flexibility.

```
$ gedit $WORK/meta-test/conf/layer.conf
# We have a conf and classes directory, add to BBPATH
BBPATH .= ":${LAYERDIR}"
# We have recipes-* directories, add to BBFILES
BBFILES += "${LAYERDIR}/recipes-*/*/.bb \
           ${LAYERDIR}/recipes-*/*/.bbappend"
BBFILES += "${LAYERDIR}/recipes-*/.bb \
           ${LAYERDIR}/recipes-*/.bbappend "
BBFILE_COLLECTIONS += "meta-test"
BBFILE_PATTERN_meta-test = "^${LAYERDIR}/"
BBFILE_PRIORITY_meta-test = "6"
LAYERDEPENDS_meta-test = "core"
LAYERSERIES_COMPAT_meta-test = "dunfell"
$
```

7.2 HelloWorld

Now, I will add below.

```
$WORK/meta-test
├── COPYING.MIT
├── README
├── conf
│   └── layer.conf
└── recipes-example
```

```

├── example
│   └── example_0.1.bb
├── recipes-images
│   └── core-image-weston.bbappend
├── recipes-test
│   └── helloworld
│       ├── helloworld.bb
│       └── src
│           ├── helloworld.c
│           └── helloworld.h

```

Prepare helloworld.c and helloworld.h

```

$ cd $WORK/meta-test
$ mkdir -p recipes-test/helloworld/src

```

And prepare below files on \$WORK/meta-test/recipes-test/helloworld/src

helloworld.h	<pre> #ifndef __HELLOWORLD_H__ #define __HELLOWORLD_H__ int helloworld(void); #endif </pre>
helloworld.c	<pre> #include <stdio.h> int main() { printf("Hello, World!\n"); return 0; }; </pre>

Create \$WORK/meta-test/recipes-test/helloworld/helloworld.bb

```

DESCRIPTION = "Example Hello, World application for Yocto build."
SECTION = "examples"
DEPENDS = ""
LICENSE = "CLOSED"

FILESEXTRAPATHS_prepend := "${THISDIR}/src:"
SRC_URI = "file://helloworld.c file://helloworld.h "

S = "${WORKDIR}"

do_compile() {
    ${CC} ${CFLAGS} ${LDFLAGS} ${WORKDIR}/helloworld.c -o helloworld
}
do_install() {
    install -d ${D}/${bindir}
    install ${WORKDIR}/helloworld ${D}/${bindir}
    install -d ${D}/${includedir}
    install ${S}/helloworld.h ${D}/${includedir}
}

```

And, add it on \$WORK/meta-test/recipes-images/core-image-weston.bbappend

```

IMAGE_INSTALL_append = " helloworld helloworld-dev"

```

Now, you are ready. Execute below commands to build.

```

$ cd $WORK/build
$ bitbake helloworld -c install

```

```
$ bitbake core-image-weston
```

The Result on Target FileSystem will have below files.

```
.
└─ usr
    └─ bin
        └─ helloworld
    └─ include
        └─ helloworld.h
```

The WORKDIR will be \$WORK/build/tmp/work/aarch64-poky-linux/helloworld/1.0-r0.

When we execute below tasks, we can see the execution step on \$WORKDIR/temp/log.task_order file.

task	Description
do_patch	It executes do_fetch, do_unpack, and do_patch. It copy "helloworld.c" and "helloworld.h" to \$WORKDIR.
do_compile	It executes do_deploy_source_date_epoch_setscene, do_prepare_recipe_sysroot, do_configure and do_compile. On these steps, bitbake uses \$WORKDIR/recipes-sysroot, and \$WORKDIR/recipes-sysroot-native.
do_install	It generates, <pre>\$WORKDIR/image └─ usr └─ bin └─ helloworld └─ include └─ helloworld.h</pre>
do_rootfs of weston-core-image	It copy "helloworld" and "helloworld.h" on \$WORK/build/tmp/h3ulcb-poky-linux/core-image-weston/1.0-r0/rootfs/usr. Note:You have to add below on core-image-weston.bbappend. <pre>IMAGE_INSTALL_append = " helloworld helloworld-dev"</pre>

To see the variable, use below example

```
bitbake helloworld -e | grep ^WORKDIR=
```

It will show below strings

variables	value
PN	helloworld
WORKDIR	\$WORK/build/tmp/work/aarch64-poky-linux/\$PN/1.0-r0
S	\$WORKDIR
B	\$WORKDIR
D	\$WORKDIR/image
FILESEXTRAPATHS	"\$WORK/meta-test/recipes-test/helloworld/src:__default:"
\${bindir}	/usr/bin
\${includedir}	/usr/include

To see the \${CC}, \${CFLAGS}, and \${LDFLAGS}, add some echo commands.

```
do_compile() {
    echo "CC=" ${CC}
    echo "CFLAGS=" ${CFLAGS}
    echo "LDFLAGS=" ${LDFLAGS}
    ${CC} ${CFLAGS} ${LDFLAGS} ${WORKDIR}/helloworld.c -o helloworld
}
```

}

And open \$WORKDIR/temp/log_do_compile

variable	value strings
CC	aarch64-poky-linux-gcc -march=armv8-a -mtune=cortex-a57.cortex-a53 -fstack-protector-strong -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysroot=\$WORKDIR/recipe-sysroot
CFLAGS	-O2 -pipe -g -feliminate-unused-debug-types -fmacro-prefix-map=\$WORKDIR=/usr/src/debug/helloworld/1.0-r0 -fdebug-prefix-map=\$WORKDIR=/usr/src/debug/helloworld/1.0-r0 -fdebug-prefix-map=\$WORKDIR/recipe-sysroot= -fdebug-prefix-map=\$WORKDIR/recipe-sysroot-native=
LDFLAGS	-Wl,-O1 -Wl,--hash-style=gnu -Wl,--as-needed -fstack-protector-strong -Wl,-z,relro,-z,now

8 META-VSPAUTO

Throughout my project, I developed 6 video bridge device drivers, over 10 device trees, specific patches for new V4L2 commands, driver modifications and libraries for VSP extended functionalities, as well as various sample codes and utility scripts. Initially, these deliverables were provided directly to customers to integrate into their own codebases.

While a few customers managed the integration successfully, the majority struggled with patch failures, leading to repetitive inquiries and heavy technical support demands.

To resolve this bottleneck, I designed and created a custom **new meta-layer**. By simply adding this meta-layer to their bblayers.conf, customers can now seamlessly integrate all the features, drivers, and configurations I developed in a single build step. This has significantly streamlined the deployment process and reduced support overhead.

Below is a detailed breakdown of the specific functionalities added for each component.

8.1 copy executable scripts

Filename	Path
720p30.sh	/home/root
setup_script.sh	/usr/local/bin
autoexec.sh	/etc/init.d

```
DESCRIPTION = "VSP Auto Applications"
LICENSE = "CLOSED"
SRC_URI = " \
    file://720p30.sh \
    file://setup_script.sh \
    file://autoexec.sh \
"
S = "${WORKDIR}"

RDEPENDS_${PN} += "bash"

do_install() {
    install -d ${D}/home/root
    cp ${S}/720p30.sh ${D}/home/root
    install -d ${D}/usr/local/bin
    cp ${S}/setup_script.sh ${D}/usr/local/bin
    install -d ${D}/etc/init.d
    cp ${S}/autoexec.sh ${D}/etc/init.d
}
FILES_${PN} += "/home/root"
FILES_${PN} += "/usr/local/bin"
FILES_${PN} += "/etc/init.d"
```

The setup_script.sh and autoexec.sh using the Bash. Add "RDEPENDS_\${PN} += "bash".

It want to install on "/home/root", "usr/local/bin" and "/etc/init.d" those are not defined on the default FILES_\${PN}. Add directory paths on FILES+\${PN} variable.

8.2 compile simple application program

It is to compile rcar-vin.c and install on /home/root with '0755' execution mode.

```
SUMMARY = "rcar-vin"
DESCRIPTION = "A test application to read VIN."
LICENSE = "CLOSED"
SRC_URI = "file://rcar-vin.c"
S = "${WORKDIR}"
do_compile() {
    ${CC} -c -O2 rcar-vin.c
    ${CC} ${LDFLAGS} -o rcar-vin rcar-vin.o
}
do_install() {
    install -d ${D}/home/root
    install -m 0755 ${S}/rcar-vin ${D}/home/root
}
FILES_${PN} += "/home/root"
```

8.3 Add CFLAGS option and use the prebuilt library

CFLAG	"-I" option is to assign the header file search path. Below will search "/usr/include/libdrm" directory
LDFLAG	"-L" option is to assign the library file search path. "-l" option is to assign the needed library.
DEPENDS	Lists a recipe's build-time dependencies. These are dependencies on other recipes whose contents (e.g. headers and shared libraries) are needed by the recipe at build time.

```
SUMMARY = "DRM mode print"
DESCRIPTION = "print DRM Mode"
LICENSE = "CLOSED"

S = "${WORKDIR}"
SRC_URI = "file://modeprint.c"

TARGET_CFLAGS += " -I ${WORKDIR}/recipe-sysroot/usr/include/libdrm"

LDFLAGS = " "
LDFLAGS += " -L ${WORKDIR}/recipe-sysroot/usr/lib"
LDFLAGS += " -ldrm"
DEPENDS = " libdrm "

FILES_${PN} += "/home/root"
do_compile() {
    ${CC} ${CFLAGS} -c -O2 modeprint.c
    ${CC} ${LDFLAGS} -o modeprint modeprint.o
}
do_install() {
    install -d ${D}/home/root
    install -m 755 ${WORKDIR}/modeprint ${D}/home/root
}
FILES_${PN} += "/home/root"
```

8.4 OEMAKE and tar.gz file

To compile several files, traditional method using Makefile. Yocto provides OEMAKE to replace it. And, we can use zipped file to provide the files.

Makefile was,

```
TARGETS := rcar-8830-smpte
SRCS := rcar-main.c rcar-vin.c drm.c drm-draw.c util.c crc_reader.c crc32.c gpio.c
OBJS = $(patsubst %.c, %.o, $(SRCS))
CFLAGS := -g -O2 -I $(SDKTARGETSYSROOT)/usr/include/libdrm
CFLAGS += -O2 -DMACHINE_H3SK
CFLAGS += -Wformat-overflow=0
LDFLAGS := -L $(SDKTARGETSYSROOT)/usr/lib -ldrm

DESTDIR=
BINDIR=/home/root

$(TARGETS) : $(OBJS)
    $(CC) $(LDFLAGS) -o $@ $^
%.o:%.c
    $(CC) $(CFLAGS) -c -o $@ $<
all: $(TARGETS)

clean:
    -rm -f *.o
    -rm -f $(TARGETS)
install: $(TARGETS)
    mkdir -p $(DESTDIR)/$(BINDIR)
    install -m 0755 $< $(DESTDIR)/$(BINDIR)
```

rcar-8830-smpte.bb is

```
SUMMARY = "rcar-8830-smpte with Makefile"
DESCRIPTION = "8830 FrameCRC generator"
LICENSE = "CLOSED"

S = "${WORKDIR}/rcar-8830-smpte"
SRC_URI = "file://rcar-8830-smpte.tar.gz"

CFLAGS += " -DMACHINE_H3SK"
LDFLAGS = " "
LDFLAGS += " -L ${WORKDIR}/recipe-sysroot/usr/lib"
LDFLAGS += " -ldrm"
EXTRA_OEMAKE = " \
    'CC=${CC}' \
    'RANLIB=${RANLIB}' \
    'AR=${AR}' \
    'CFLAGS=${CFLAGS}' \
    -I${S}/. \
    -I ${WORKDIR}/recipe-sysroot/usr/include/libdrm -DWITHOUT_XATTR ' \
    'LDFLAGS=${LDFLAGS}' \
    'BUILDDIR=${S}' \
"
# TARGET_LDFLAGS += " -ldrm"
```

```

DEPENDS = " libdrm "
RPROVIDES_${PN} += "libdrm"
RDEPENDS_${PN} += "libdrm"

# replace "BINDIR=${bindir}" to "BINDIR=home/root"
do_compile() {
    oe_runmake all DESTDIR=${D} BINDIR=home/root SBINDIR=${sbindir} \
                MANDIR=${mandir} INCLUDEDIR=${includedir}
}
do_install() {
    oe_runmake install DESTDIR=${D} BINDIR=home/root SBINDIR=${sbindir} \
                MANDIR=${mandir} INCLUDEDIR=${includedir}
}
FILES_${PN} += "/home/root"

```

	Description
RDEPENDS	Lists runtime dependencies of a package. These dependencies are other packages that must be installed in order for the package to function correctly.
RPROVIDES	A list of package name aliases that a package also provides. These aliases are useful for satisfying runtime dependencies of other packages both during the build and on the target (as specified by RDEPENDS).

8.5 OECPMAKE for CMake

rcar-vin2drm.bb

```

SUMMARY = "rcar-vin2drm with cMake"
DESCRIPTION = "copy VIN to DRM"
LICENSE = "CLOSED"

inherit cmake

S = "${WORKDIR}/rcar-vin2drm"
SRC_URI = "file://rcar-vin2drm.tar.gz"
DEPENDS = " libdrm "

do_install() {
    install -d ${D}/home/root
    install -m 755 ${B}/utest/rcar-vin2drm ${D}/home/root
}
FILES_${PN} += "/home/root"
EXTRA_OECMAKE += " -DCMAKE_SYSROOT=${STAGING_DIR_TARGET}"

```

CMakeLists.txt

```

cmake_minimum_required (VERSION 2.8)
project(rcar-vin2drm)

# ...some external headers having no corresponding "FindXXX.cmake" facilities
SET(DRM_INCLUDE_DIR "${CMAKE_SYSROOT}/usr/include/drm")
#SET(GLIB_INCLUDE_DIR "${CMAKE_SYSROOT}/usr/include/glib-2.0" "$
{CMAKE_SYSROOT}/usr/lib/glib-2.0/include" "${CMAKE_SYSROOT}/usr/lib64/glib-
2.0/include" "/usr/lib/x86_64-linux-gnu/glib-2.0/include")

```

```
# ...common libraries
#SET(COMMON_LIBRARIES "pthread" "dl" "z" "m")
#SET(GLIB_LIBRARIES "gobject-2.0" "glib-2.0" "gmodule-2.0" "gthread-2.0")
#SET(MMNGR_LIBRARIES "mmngr" "mmngrbuf")
#SET(WNDSYS_LIBRARIES "drm" "udev")
SET(WNDSYS_LIBRARIES "drm" )

# ...force stripping of the output files during installation
SET(CPACK_STRIP_FILES TRUE)

# ...unit-test application
add_subdirectory("utest")
```

CMakeLists.txt on utest

```
cmake_minimum_required (VERSION 2.8)
project("rcar-vin2drm")

if(CMAKE_BUILD_TOOL MATCHES "(msdev|devenv|nmake|MSBuild)")
    add_definitions("/W2")
else()
    add_definitions("-Wall -Wextra -Werror -Wno-unused-parameter
-Wno-unused-localtypedefs -Wno-unused-variable -Wno-unused-but-set-variable
-Wno-strict-aliasing")
endif()

# ...add sources
file(GLOB APP_C_SRC "drm.c" "drm-draw.c" "rcar-vin.c" "util.c" "rcar-main.c")

include_directories(${DRM_INCLUDE_DIR})

add_executable(rcar-vin2drm ${APP_C_SRC})
target_link_libraries(rcar-vin2drm ${WNDSYS_LIBRARIES})

#
# ${COMMON_LIBRARIES}
set_target_properties(rcar-vin2drm PROPERTIES SKIP_BUILD_RPATH ON)

# ...installation
if (NOT BINARY_DIR)
    set(BINARY_DIR ${CMAKE_CURRENT_SOURCE_DIR}/../output)
endif()

message(STATUS "Installation directory: ${BINARY_DIR}")

install(TARGETS rcar-vin2drm DESTINATION ${BINARY_DIR})
```

8.6 Add new Device driver and Device Tree

```
meta-vspauto
+- recipes-kernel
  +- core-image-weston.bbappend
```

```
+ linux-renesas_5.10.bbappend
+ linux-renesas
+ r8a779m1-ulcb-9974.dts
+ vsp-001-drm.patch
+ vsp-002-v4l2.patch
+ vspauto.cfg
```

8.6.1 Add new Device Tree

At first, if you add your device tree on \$WORK/meta-renesas/meta-rcar-gen3/conf/machine/h3ulcb.conf file, remove it. We will add it on our new meta layer.

```
KERNEL_DEVICETREE += "renesas/r8a778m1-ulcb-yours.dtb"
```

On linux-renesas_5.10.bbappend, add DTS/DTB related strings.

```
..
SRC_URI_append = " file://r8a779m1-ulcb-9974.dts"
..
do_update_vspauto_dtbs() {
    install -d ${S}/arch/arm64/boot/dts/renesas
    install ${WORKDIR}/ r8a779m1-ulcb-9974.dts ${S}/arch/arm64/boot/dts/renesas
    ..
}
FILES_${PN} += "${S}/arch/arm64/boot/dts/renesas

addtask do_update_vspauto_dtbs after do_patch before do_kernel_configm

KERNEL_DEVICETREE += " renesas/ r8a779m1-ulcb-9974.dtb"
...
```

8.2.2 Prepare kernel patch file

```
$KERNEL_LOC/drivers/media/i2c/
+- Kconfig
+- Makefile
+- raa279974.c
+- raa279974.h
```

First and very important thing you have to do is a backup.

```
$ cd $KERNEL_LOC
$ cd ..
$ cp -rf kernel-source kernel-modified
```

Second thing is prepare 'a' directory with your original kernel-source.

```
$ cd $KERNEL_LOC
$ cd ..
$ cp -rf kernel_org a
$ cp -rf a b
```

If you did not save the kernel_org file before, we can use below commands. In this case your modified code will be disappeared.

```
$ cd $WORK/build
$ bitbake linux-renesas -c cleanall
$ bitbake linux-renesas -c fetch
$ bitbake linux-renesas -c patch
$ cd $KERNEL_LOC/..
$ cp -rf kernel-source kernel_org
$ cp -rf kernel_org a
```

Third step is a prepare 'b', and manually merge your modified code manually. I used a 'meld' tool.

```
$ cp -rf a b
$ meld
```

Compare kernel_modified and 'b' and then update manually.

Final step is, generates the patch file. Use "-ruN" option.

```
$ diff -ruN a/ b/ > vsp-002-vl42.patch
$ mkdir -p $WORK/meta-vspauto/recipes-kernel/linux-renesas
$ cp vsp-002-vl42.patch $WORK/meta-vspauto/recipes-kernel/linux-renesas
$
```

On linux-renesas_5.10.bbappend, add patch

```
SRC_URI_append = " file://vsp-002-vl42.patch"
```

8.6.3 add .config

```
$ cd $WORK/meta-vspauto
$ gedit vspauto.cfg
CONFIG_VIDEO_RAA279974=y
CONFIG_BK_PR_ERR_MESSAGE=y
<You can add more CONFIG, for example CONFIG_USB_SERIAL=y and
CONFIG_USB_SERIAL_PL2303=y>
```

On linux-renesas_5.10.bbappend, add cfg file

```
SRC_URI_append = " file://vspauto.cfg"
```

8.7 Add Kernel module

```
meta-vspauto/
+- core-image-weston.bbappend
+- kernel-module-rcar-imr.bb
+- rcar_imr-files/
  +- Makefile
  +- rcar-imr.c
  +- rcar-imr.h
```

8.7.1 kernel-module-rcar-imr.bb

```
DESCRIPTION = "RCAR IMR-LX4 Driver"
require include/rcar/gen3-modules-common.inc
```

```

LICENSE = "CLOSED"
inherit modules
FILESEXTRAPATH_prepend := "${THISDIR}/rcar0imr-files:"
DEPEND = "linux-renesas"
PN = "kernel-module-rcar-imr"
PR = "r0"
SRC_URI = file://Makefile file::rcar-imr.c file::rcar-imr.h
S = "${WORKDIR}"

KERNEL_MODULE_PACKAGE_SUFFIX = ""

do_compile() {
    #cd ${S}
    make all
}

do_install() {
    install -d ${D}/lib/modules/${KERNEL_VERSION}/extra/
    install -m 644 ${S}/rcar/imr.ko ${D}/lib/modules/${KERNEL_VERSION}/extra/
}

PACKAGES = " ${PN} ${PN}-dev "
FILES_${PN} = " /lib/modules/${KERNEL_VERSION}/extra/rcar-imr.ko "
RPROVIDES_${PN} += "kernel-modules-rcar-imr"
KERNEL_MODULE_AUTOLOAD_append = " rcar-imr"

```

8.7.2 add on core-image-weston.bbappend

```

IMAGE_INSTALL_append = " kernel-module-rar-imr kernel-module-rcar-imr-dev"

```

8.7.3 Others kernel modules that need an autoload.

File	Contents
kernel-module-mmngr.bbappend	KERNEL_MODULE_AUTOLOAD += "mmngr"
kernel-module-mmngrbuf.bbappend	KERNEL_MODULE_AUTOLOAD += "mmngrbuf"
kernel-module-vspmif.bbappend	RPROVIDES_\${PN}:remove = "kernel-module-vspm-if" RPROVIDES_\${PN} = "kernel-module-vspm_if" KERNEL_MODULE_AUTOLOAD_append = "vspm_if"

8.8 Add USB3.0 firmware

Add USB3.0 features on linux-renesas_5.10.bbappend

```

# Configuration for USB3.0 on ulcb-CeX extension board
MACHINE_FEATURES_append = " usb3"

```

8.9 Add modified libraries

core-image-weston.bbappend

```

meta-vspauto/

```

```

+- recipes-graphics/
| +- wayland/
| | +- libgbm/
| | | +- vspauto-libgbm-1.0.patch
| | +- wayland/
| | | +- vspauto-wayland-1.18.0.patch
| | +- Weston/
| | | +- vspauto-weston-8.0.0.patch
| | +- libgbm.bbappend
| | +- wayland_1.18.0.bbappend
| | +- Weston_8.0.0.bbappend
+- recipes-multimedia/
  +- v4l2apps/
    +- v4l-utils/
      +- vspauto-v4l-utils-1.18.1.patch
    +- v4l2-utils_1.18.1.bbappend

```

v4l2-utils_1.18.1.bbappend

```

SUMMARY = "v4l2app patch"
DESCRIPTION = "patch on v4l2apps"
LICENSE = "GPLv2 & LGPLv2.1"

FILESEXTRAPATHS_prepend := "${THISDIR}/${PN}:/:"
SRC_URI_append = " file://vspauto-v4l-utils-1.18.1.patch"

```

8.9 Add Layer

```

$WORK/
+- meta-vspauto/
| +- COPYING.MIT
| +- README
| +- conf/
| | +- layer.conf
| +- IPL/
| +- recipes-graphics/
| +- recipes-kernel/
| +- recipes-multimedia/
| +- recipes-vspapp/
+- build
  +- conf
  | +- bblayers.conf
  | +- ...
  +- tmp
  +- ...

```

layer.conf

```

BBPATH .= ":${LAYERDIR}"
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
            "${LAYERDIR}/recipes-*/*/*.bbappend \
            "${LAYERDIR}/recipes-*/*/*.bb \
            "${LAYERDIR}/recipes-*/*/.bbappend "
BBFILE_COLLECTIONS += "meta-vspauto"

```

```
BBFILE_PATTERN_meta-vspauto = “^${LAYERDIR}/”  
BBFILE_PRIORITY_meta-vspauto = “7”  
LAYERDEPENDS_meta-vspauto = “core”  
LAYERSERIES_COMPACT_meta-vapsuto = “dunfell”
```

8.9.1 Last things to do.

To add meta-vspauto, add below line on \$WORK/build/conf/bblayers.conf

```
..  
BBLAYERS += “ ${TOPDIR}/../meta-vspauto
```

And then, re-generate images and SDK

```
$ bitbake core-image-weston  
$ bitbake core-image-weston-sdk -c populate_sdk
```

Reference

	Location
Yocto	https://docs.yoctoproject.org/overview-manual/intro.html
Bitbake	https://docs.yoctoproject.org/bitbake/2.18/index.html
BSP	https://docs.yoctoproject.org/bsp-guide/index.html
Kernel	https://docs.yoctoproject.org/kernel-dev/index.html
VSPAuto	https://github.com/vspauto/VideoDrivers